

パイプライン処理

ISHI会

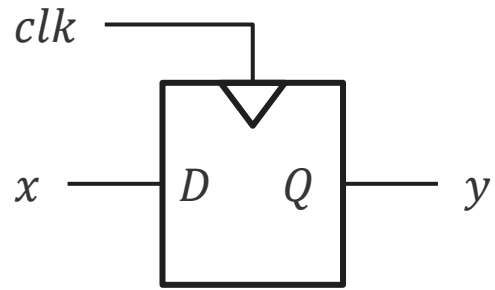
フルデジタルで音声を操るためのロジック回路 6回目

目次

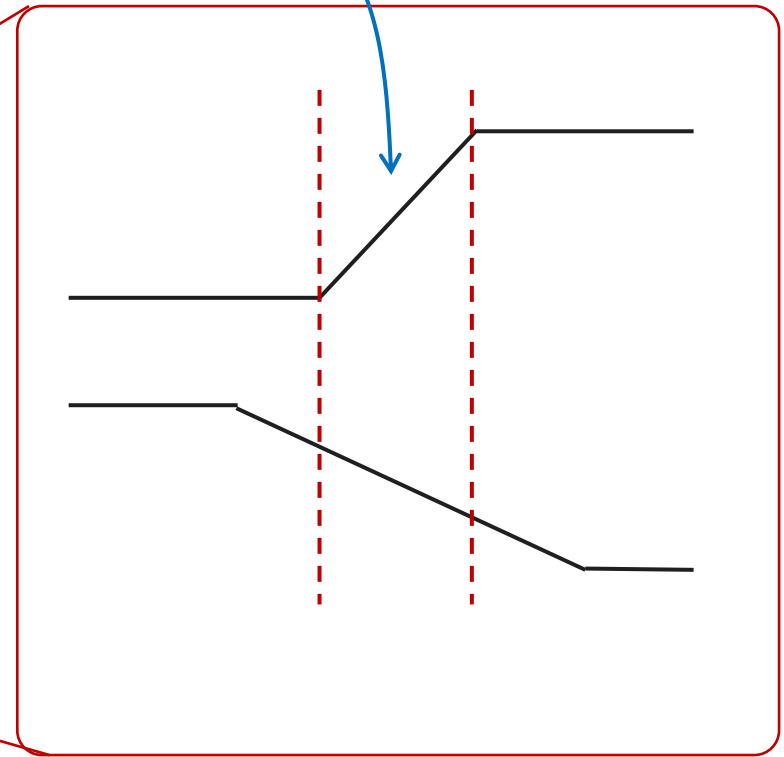
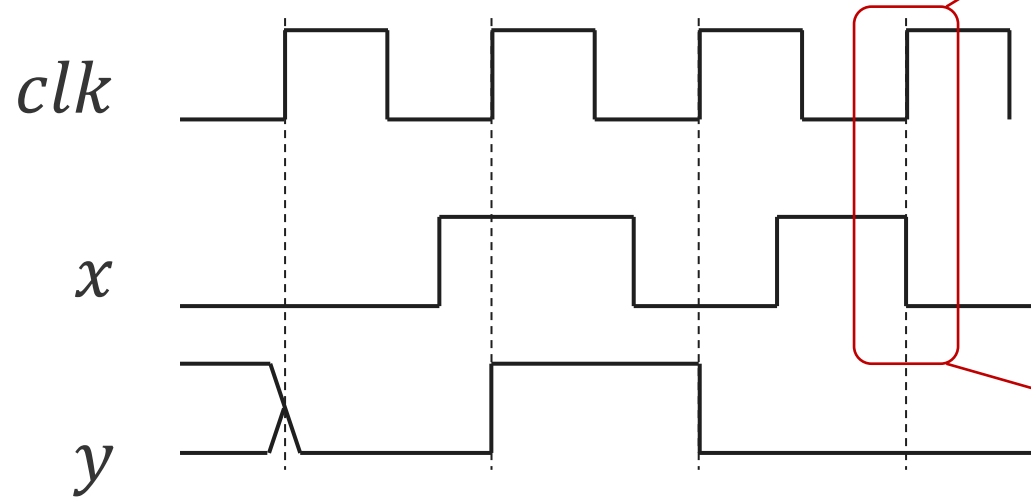
- タイミングの問題
- パイプライン処理
- Direct Digital Synthesis
- これまでのいろいろのくみあわせ

タイミングの問題

D-FFがデータを取得するときのタイミング

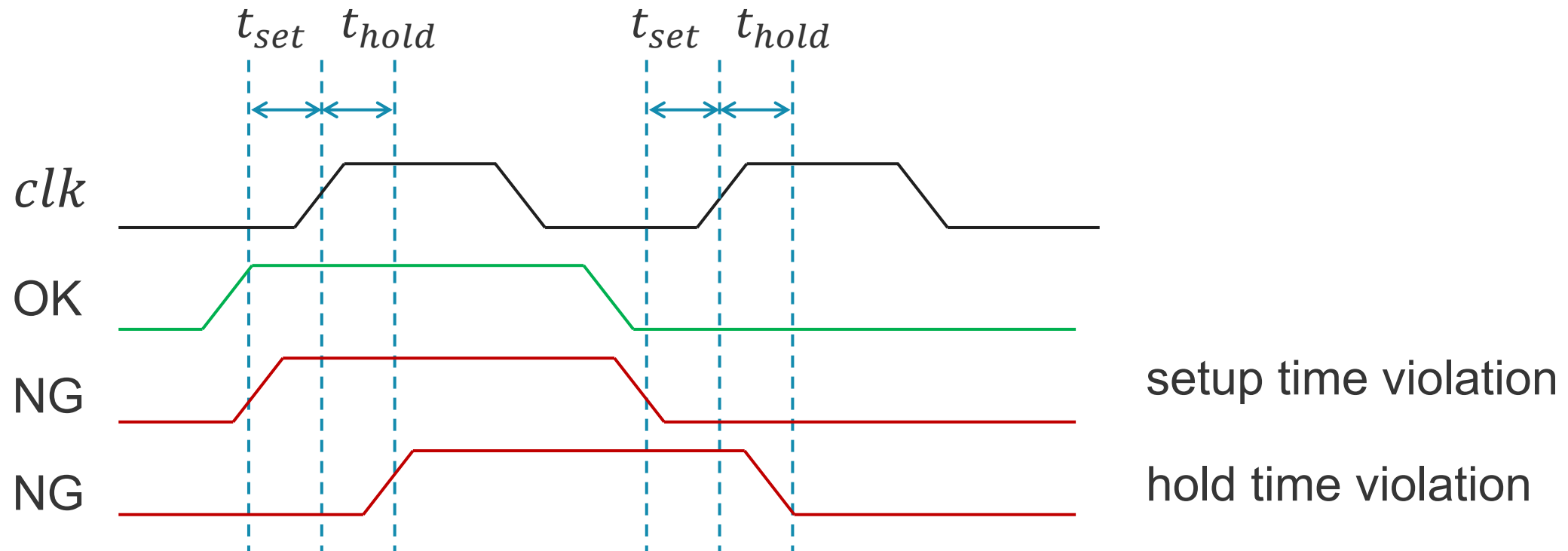


この間のどこの値が取得されるのだろうか？

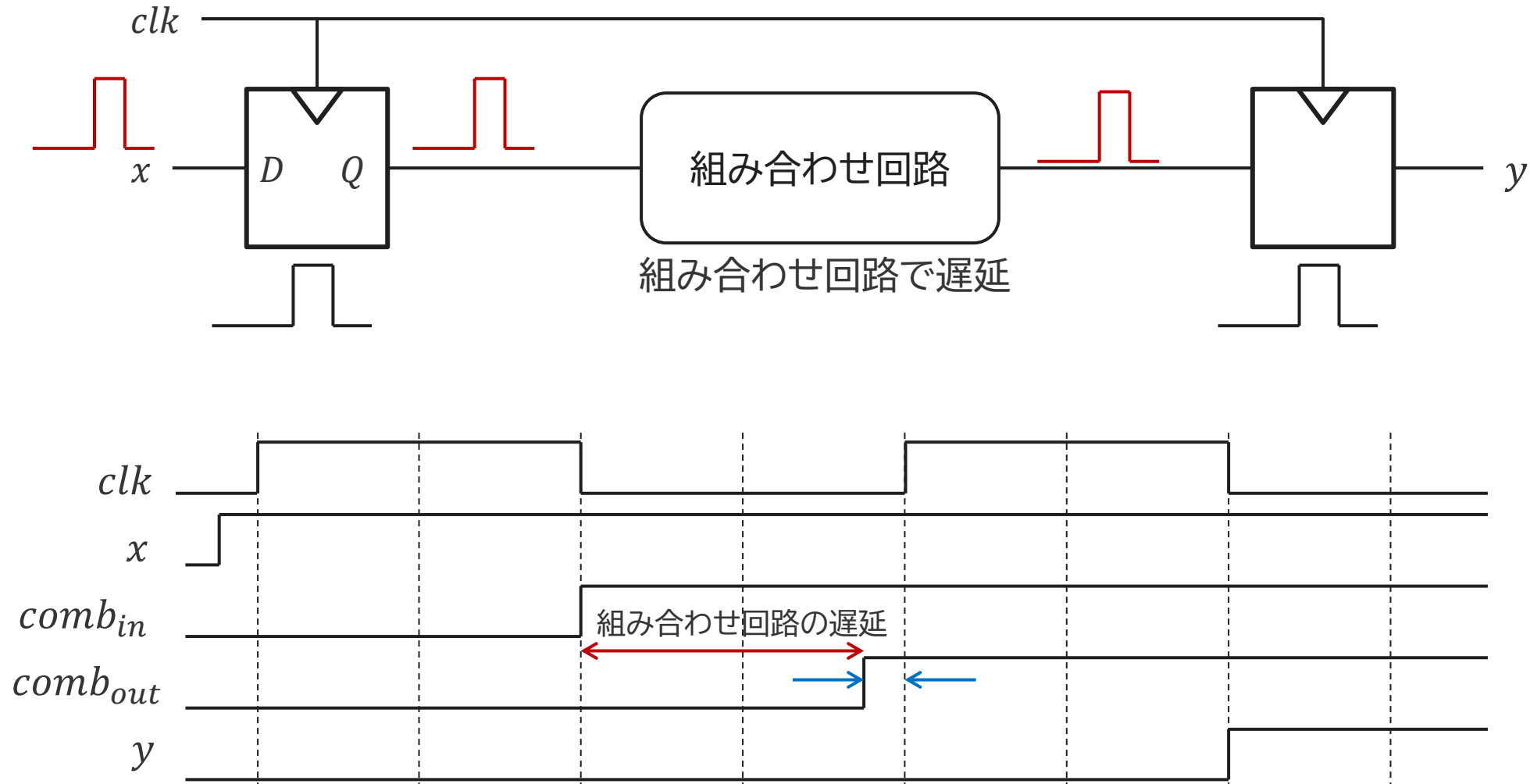


D-FFを使うときのルール

- クロック立ち上がりの前後は入力の値を安定にしておく必要がある。
- ✓ クロック立ち上がり前に安定しているべき時間→セットアップタイム
 - ✓ クロック立ち上がり後に安定しているべき時間→ホールドタイム

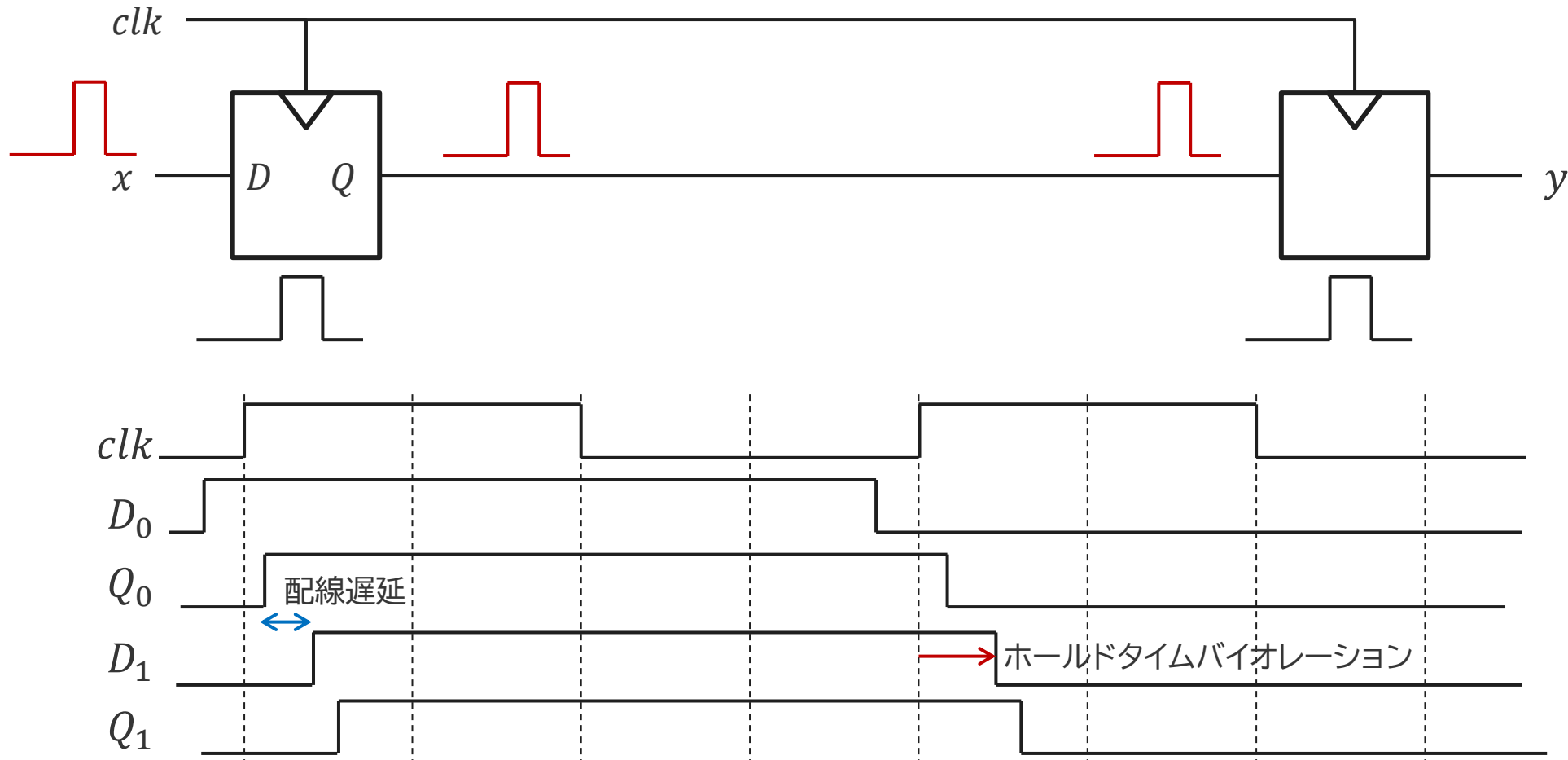


セットアップタイムバイオレーション



組み合わせ回路の遅延がクロック周期に近くなるとセットアップタイムを満たさなくなる

ホールドタイムバイオレーション

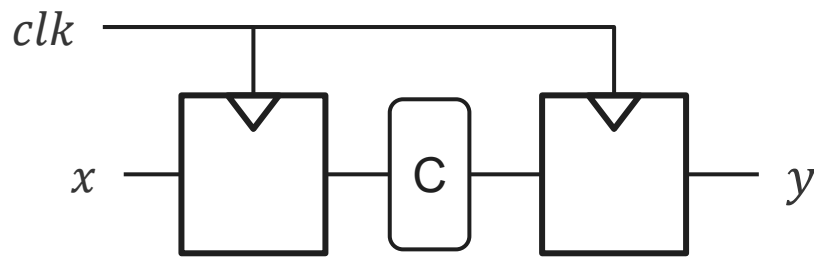


データ到達が非常に早い場合にホールドタイムを満たせなくなる

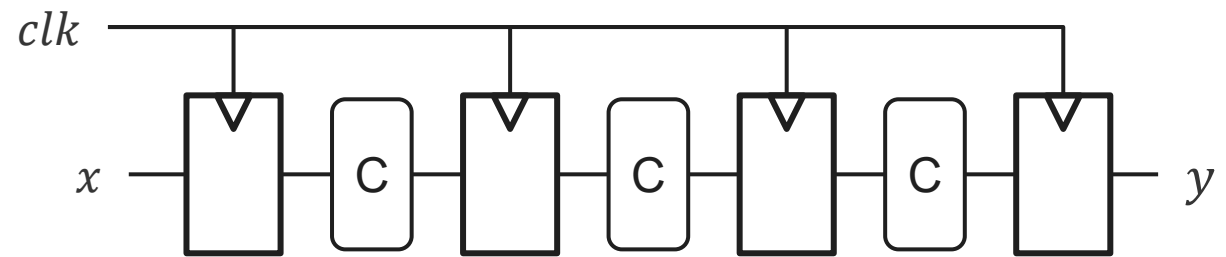
パイプライン処理

クロック周波数を高くするには？

- 単位時間あたりのデータ処理量を増やすにはクロック周波数を高くするのが簡単
 - ✓ 高い周波数はセットアップタイムを満たすのが難しくなる
- 組み合わせ回路の遅延を短くするのが常套手段
 - 1つの組み合わせ回路を複数に分割することで遅延を短くする
 - レイテンシは増える

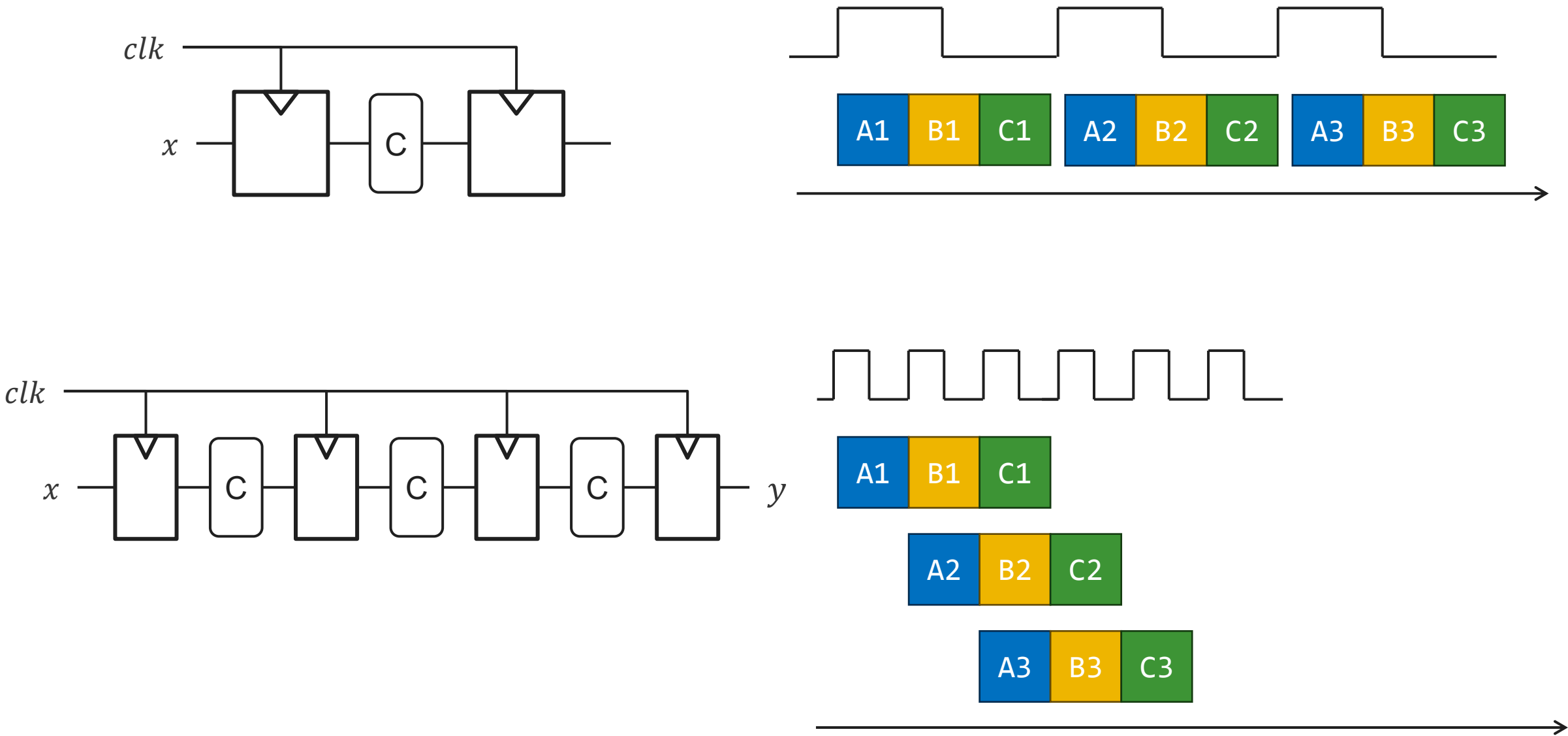


長大な組み合わせ回路



複数段の簡単な組み合わせ回路

パイプライン処理



Direct Digital Synthesis

サイン波を出力する

あなたは、急に $\sin(0.1\pi)$ を求めなければならなくなっていました。

しかし、今ここには関数電卓も三角関数表也没有ありません。

最低でも4桁の精度は必要だと言われています。どうやりますか？

先人は常に偉大です。たくさんのアルゴリズムが考えられてきました。

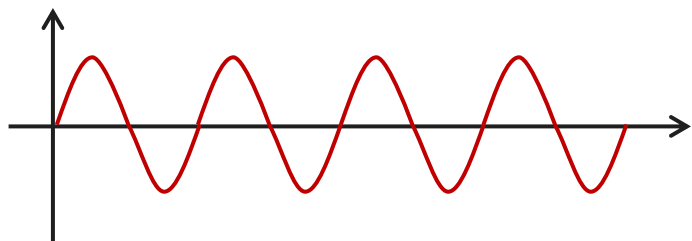
ここではテイラー展開とCORDICアルゴリズムというのを挙げておきます。

$$\text{sin関数のテイラー展開} \quad \sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

理論的にはある時間の位相に対応する振幅を高速に計算、出力することで直接(**Direct**)、デジタル(**Digital**)に波形を合成(**Synthesis**)できる。

DDSの正弦関数の計算は？

サイン波は周期関数



先に必要なデータを計算しておけば、
あとはそれを繰り返し読み出す



Look Up Table

位相	値
0	0
0.01π	0.0054803
0.02π	0.00109607
$\vdots$	
1.98π	0.108298
1.99π	0.05477591

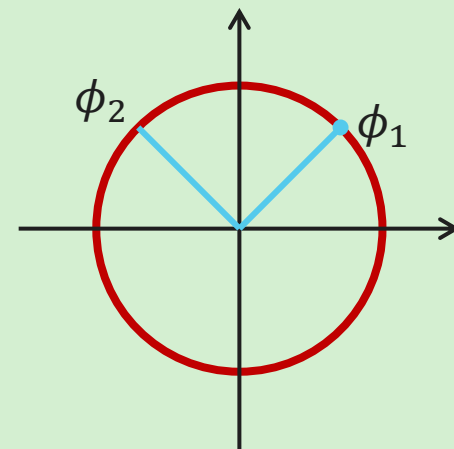
Look Up Table

位相に対応する振幅を先に計算して表を作る

位相と周波数

位相 (phase)

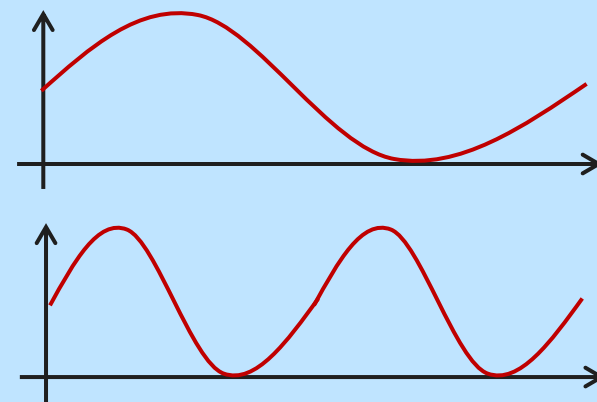
繰り返される現象のなかで、一つの周期中のある特定の位置を表す量



周波数 (frequency)

繰り返される現象が単位時間あたりに何回起こるのかを表す量。

ある時間に位相がたくさん進んだとすると、その事象は周波数が高いと呼ぶ

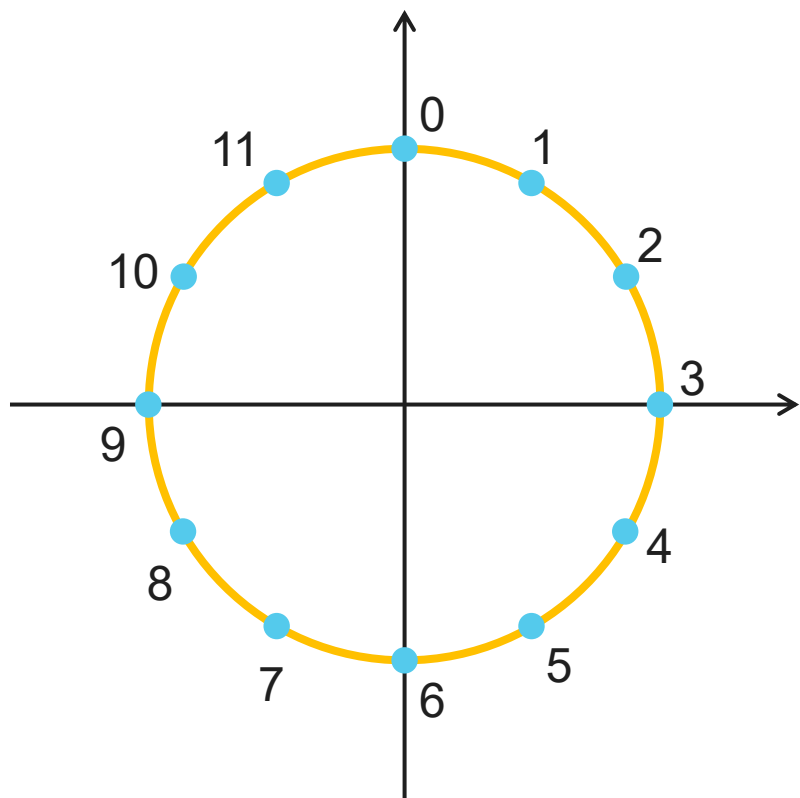


デジタル空間における位相の表現

アナログでは、位相は連続で自由



デジタルでは、位相も離散化

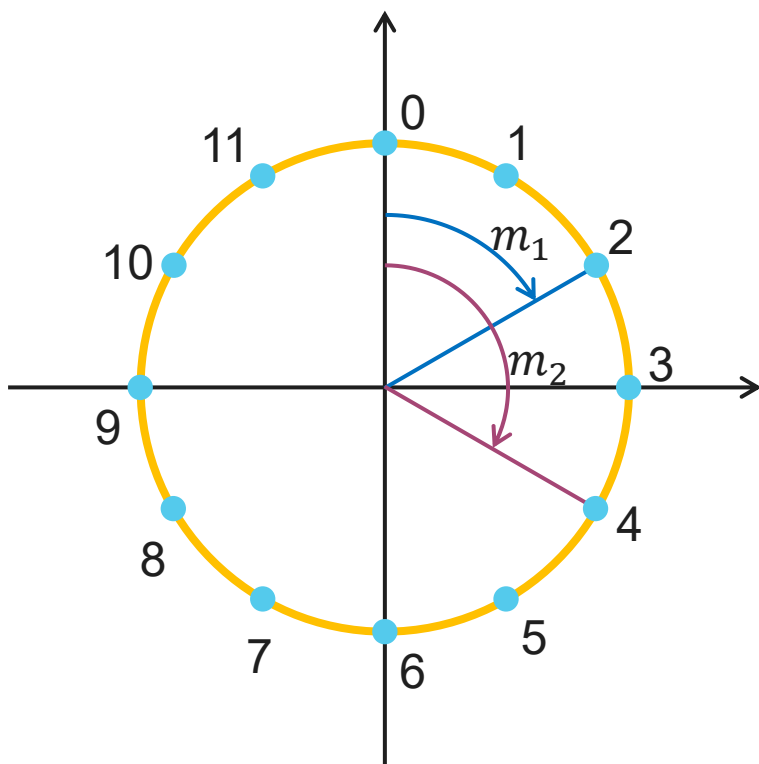


- 位相の離散化をどの程度まで行うかはシステムの要求性能と出力アナログ回路の性能から決まる。
- 市販のDDS-ICでは16～48ビット程度の離散化。
- 位相の分割数から、Look Up Tableの要素数が決定。
- デジタルの世界では、 2^n 分割になるようにnを選択。

デジタル空間における周波数

周波数は位相ステップ m をどの程度取るかにより決まる。

位相ステップ m は一回の位相の更新で位相を進める量を表す



位相の更新周期を T_c 、位相分割数を M とすると

m_1 の位相ステップでは
6サイクルで一周

m_2 の位相ステップでは
3サイクルで一周

$$\text{周期 } T_1 = T_c \times \frac{M}{m_1} = 6T_c$$

$$\text{周期 } T_2 = T_c \times \frac{M}{m_2} = 3T_c$$

$$\text{周波数 } f_1 = \frac{1}{T_1} = \frac{1}{6} f_c$$

$$\text{周波数 } f_2 = \frac{1}{T_2} = \frac{1}{3} f_c$$

生成される波形の周波数は位相ステップの逆数と更新周波数の積になる

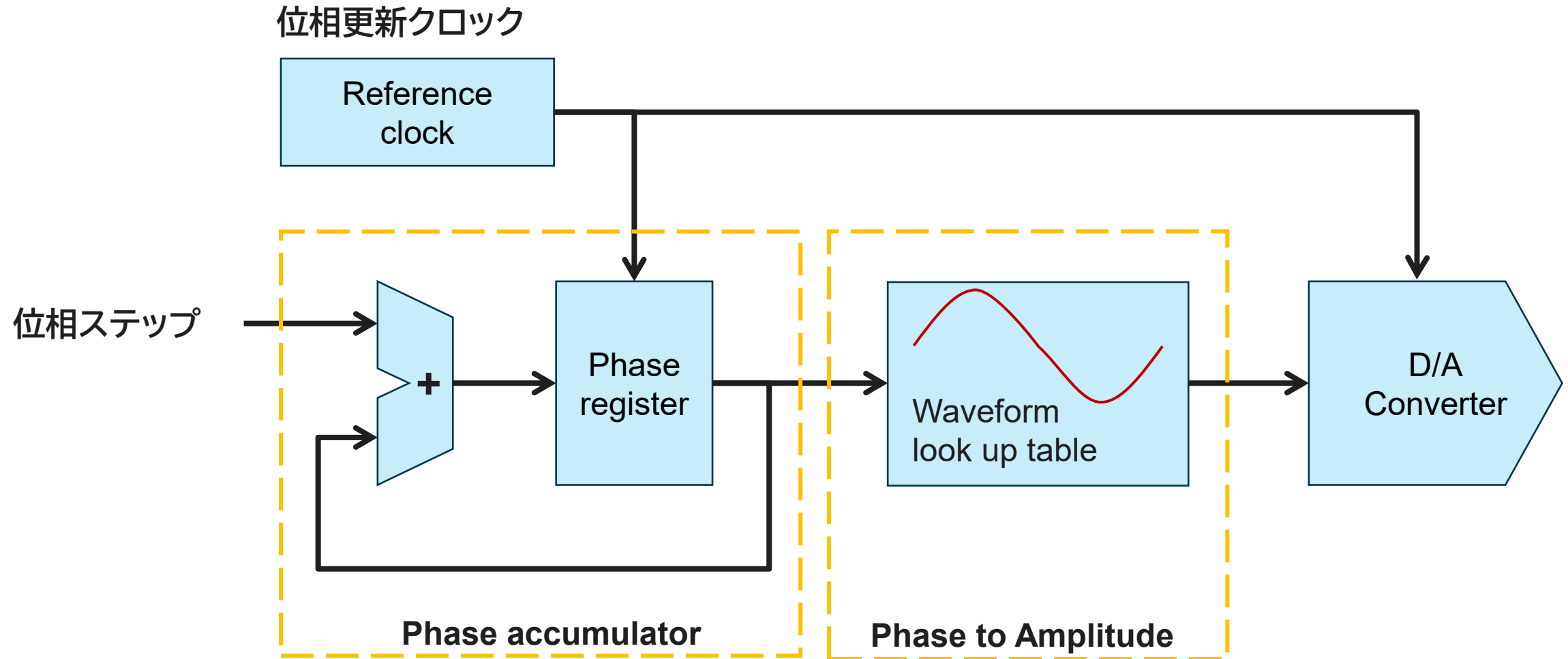
DDSを実現するには？

- ✓ DDSでは波形テーブルを再生することで信号を作り出す
- ✓ 波形テーブルには位相とその時の振幅を記録
- ✓ 波形テーブルの要素数は位相の分割数決まる
- ✓ 位相を更新するための周波数 f_c (周期 T_c)のクロック
- ✓ 1回の更新で位相を進める量を表すステップ数 m
- ✓ デジタルデータをアナログ信号にするD/Aコンバータ

これらを実現するハードウェア構成を考える

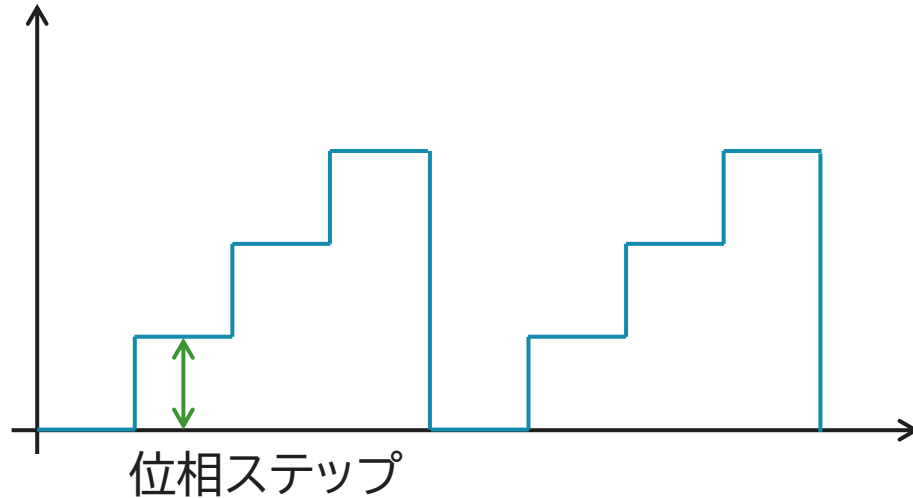
Direct Digital Synthesizerの構成

18

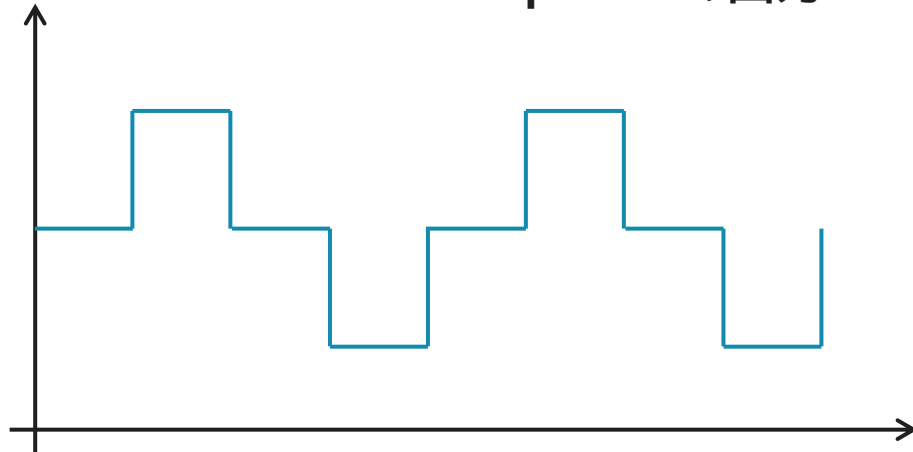


DDSの動作

位相アキュムレータの累積値

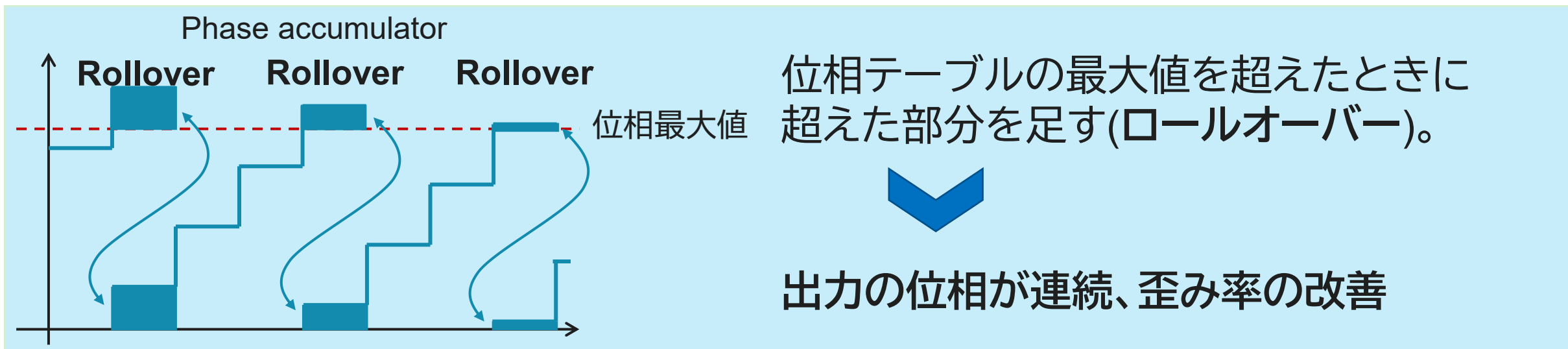
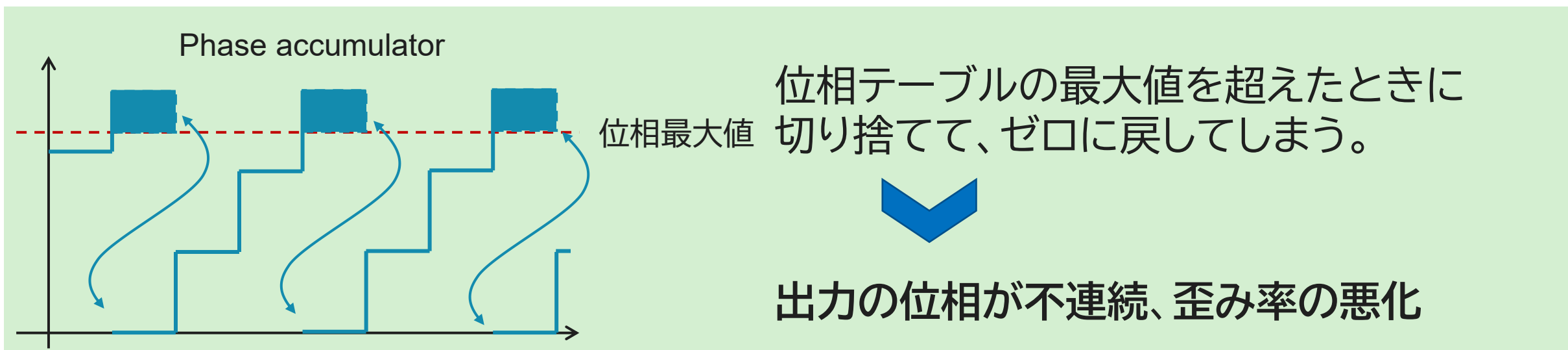


Waveform look-up tableの出力

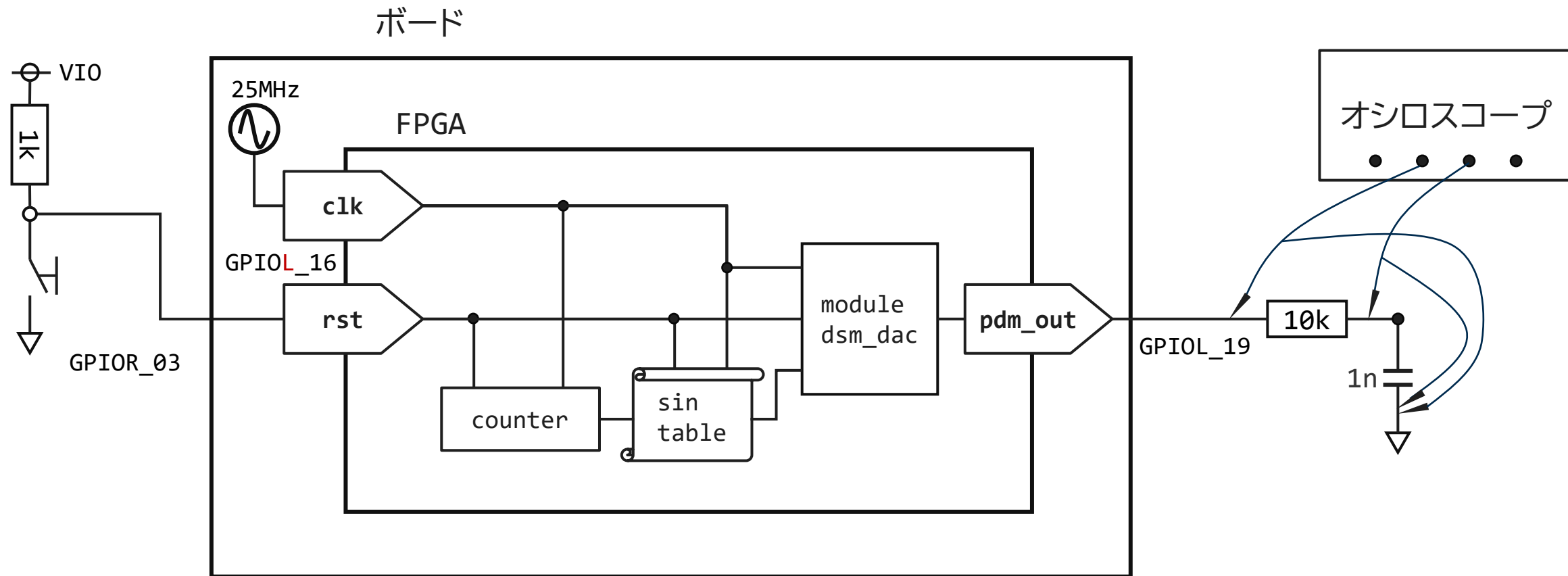


- ✓ 位相アキュムレータは常に線形に増加
- ✓ フルカウントになったらゼロに戻る
- ✓ 波形テーブルは位相アキュムレータの値に対応した振幅をD/Aコンバータに出力

位相ステップが位相分割数の約数にならないとき



サイン波を出力するDDS実装

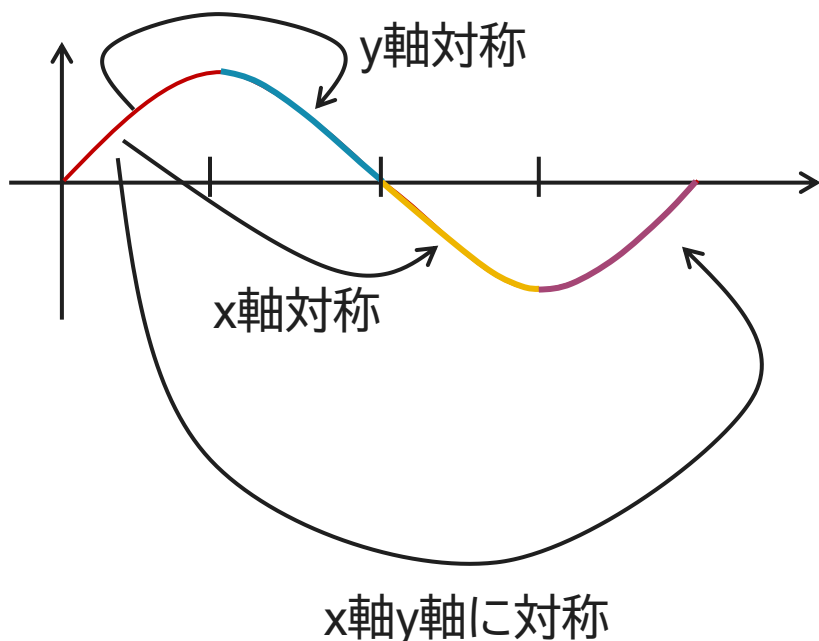


サイン波Look up tableの作り方

- テーブルの数を決める
 - 2^n の個数になるようにすると後で楽になる
- 振幅のビット数を決める
 - 大きすぎても後処理が大変、少ないと歪みが増える

振幅: 10ビット (1024段階)
テーブル件数: 10ビット (1024件)

テーブル件数をもっと減らしたい → サイン波の特徴を考える



1/4周期の波形データを持っておけば、サイン波はすべて再現できる！

サイン波Look up tableを作るコード

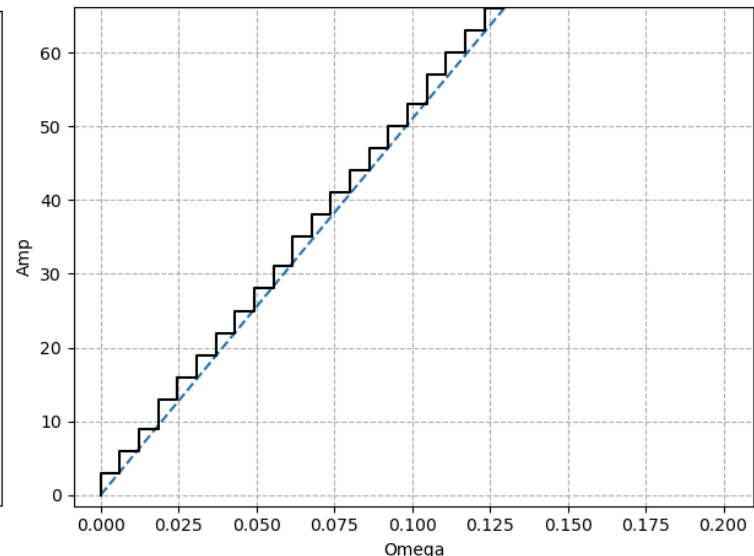
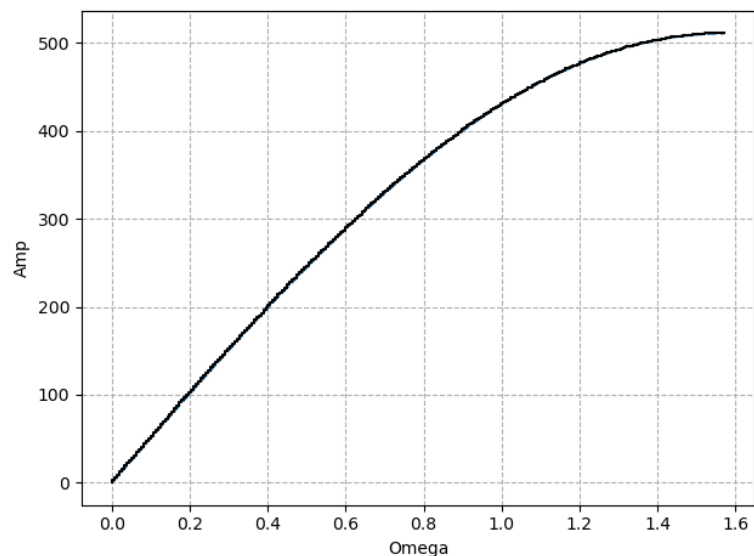
```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 if __name__ == "__main__":
5     x = np.linspace(0, np.pi/2, 256);
6     y = (np.sin(x)*511).round().astype(np.int64)
7
8     with open("sin_table.txt", "w") as f:
9         for n in y:
10             print("{0:0>10b}".format(n & 0b111111111, 'b'))
11             f.write("{0:0>10b}\r\n".format(n & 0b111111111, 'b'))
12
13 yy = np.sin(x)*511
14 fig = plt.figure()
15 ax = fig.add_subplot(111)
16 ax.plot(x, yy, ls = '-', color='C0')
17 ax.step(x, y, ls = '--', color='k')
18 ax.set_xlabel('Omega')
19 ax.set_ylabel('Amp')
20 ax.grid(ls='--')
21 fig.tight_layout()
22 plt.show()

```

テーブルを作ってファイルに出力

グラフを出力



sin_tableの実装1

ロジックを使って実装すると、FPGAのリソースが全然足りない
ので、SRAM(ROM)が推論されるように工夫をする

ファイルの内容をレジスタに読み取る
→後の記述で内蔵のSRAMが推論されるように細工する

アドレスの範囲で処理を分ける

- y軸対称の箇所は反転して読み出す
- x軸対称の箇所はそのまま読み出す

```

1 module rom_sin
2 (
3     input wire clk,
4     input wire xrst,
5     input wire [9:0] address,
6     output reg [9:0] data
7 );
8
9     reg [9:0] rom [0:255];
10    initial
11    begin
12        $readmemb("./sin_table.txt", rom);
13    end
14
15    /*
16     * 10ビットアドレスから8ビットアドレスへの変換。
17     * 上位2ビットで4領域を分ける。
18     * 00 : 0から1/2π
19     * 01 : 1/2πからπまで
20     * 10 : πから3/2πまで
21     * 11 : 3/2πから2πまで
22     */
23    reg [7:0] addr;
24    always @(*)
25    begin
26        case(address[9:8]) // 上位2ビットで条件わけ
27            2'b00: addr = address[7:0];
28            2'b01: addr = ~address[7:0];
29            2'b10: addr = address[7:0];
30            2'b11: addr = ~address[7:0];
31            default: addr = address[7:0];
32        endcase
33    end

```


sin_tableの実装2

```

35  /*
36  *  π以上の範囲では符号反転する。
37  *  一応4領域に分けて書く。
38  *  00 : 0から1/2π
39  *  01 : 1/2πからπまで
40  *  10 : πから3/2πまで
41  *  11 : 3/2πから2πまで
42  *
43  *  SRAMを推論させるために、
44  *  一旦ROMからデータを取り出して計算。
45  *  (アドレス入れてからデータ出るまで2クロック遅延)
46  *
47  */
48  reg [9:0] data2;
49  reg [1:0] addr2;
50  always @(posedge clk or negedge xrst)
51  begin
52      if (!xrst) data2 <= 10'b0;
53      else
54      begin
55          addr2 <= address[9:8];
56          data2 <= rom[addr];
57      end
58  end
59
60  always @(posedge clk or negedge xrst)
61  begin
62      if (!xrst) data <= 10'b0;
63      else
64      begin
65          case(addr2) // 上位2ビットで条件わけ
66              2'b00: data <= data2;
67              2'b01: data <= data2;
68              2'b10: data <= ~data2+1;
69              2'b11: data <= ~data2+1;
70          endcase
71      end
72  end
73
74  endmodule

```

1回遅延させることでSRAMを推論させやすくする
SRAMの操作は時間がかかるので、レジスタへの転送と実際に計算を行う部分を分ける(パイプライン化)

反転して読み出すだけだと、符号が合わないので
1/2π以上で符号を反転させる

サイン波を再生するDDSの実装

```

1 module dds_dac(
2   input wire clk,
3   input wire xrst,
4   input wire [23:0] omega, ← 位相加算量(周波数を決める)
5   output wire pdm_out);
6
7   wire signed [9:0] wave_data;
8   wire [9:0] u_wave_data;
9
10  dsm_dac uut(
11    .clk(clk),
12    .xrst(xrst),
13    .data(u_wave_data),
14    .dsm_out(pdm_out));
15
16  rom_sin sin(
17    .clk(clk),
18    .xrst(xrst),
19    .address(address[23:14]) ← サインテーブルのアドレスは上位10ビットを使用
20    .data(wave_data)
21  );
22
23  assign u_wave_data = wave_data ^ 10'h200;
24  reg [23:0] address; ← サインテーブルのアドレスは24ビット(実数部10ビット、小数部14ビット)
25  always @(posedge clk or negedge xrst)
26  begin
27    if (!xrst)
28    begin
29      address <= 24'd0;
30    end
31    else
32    begin
33      address <= address + omega;
34    end
35  end
36
37 endmodule

```

DSM-DACのインスタンス

サインテーブルのインスタンス

サインテーブルのアドレスは上位10ビットを使用

サインテーブルのアドレスは24ビット(実数部10ビット、小数部14ビット)

DDSの位相加算器

トップモジュール

```
1  ⊖ module dds_top(  
2    input wire clk,  
3    input wire xrst,  
4    output pdm_out);  
5  
6  ⊖ dds_dac dds(  
7    .clk(clk),  
8    .xrst(xrst),  
9    .omega(24'd400),  
10   .pdm_out(pdm_out));  
11  
12  endmodule  
13
```

← この値で周波数が変わる

ピンアサイン

GPIO : Instance View

			all	all	all	all	
Instance	Package Pin	Resource	I/O Bank	Alt Conn	Features	Clock Region	Pad
clk	C2	GPIOL_16	1B	GCLK	None	L1	GPIOL_16_CLK2
pdm_out	G1	GPIOL_12	1A	GCTRL	None	L0	GPIOL_12_CTRL0
xrst	A6	GPIOR_03	2A	None	None	R1	GPIOR_03

400の結果



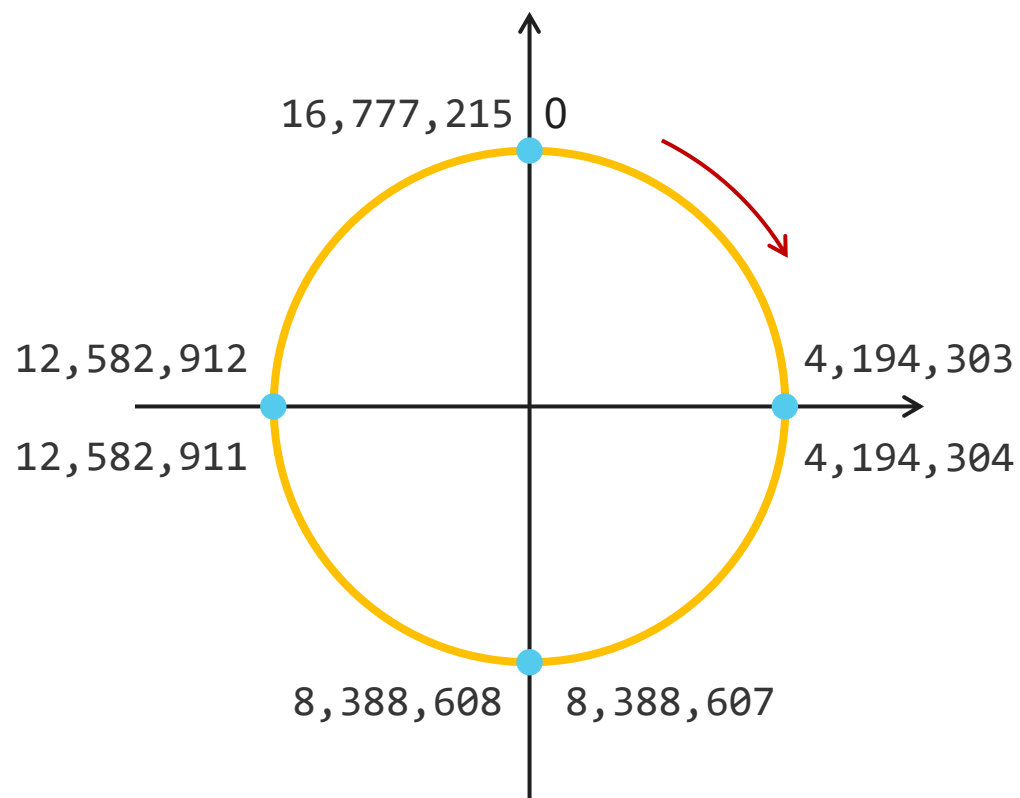
595 Hz

周波数はどのように決まるのか？

きれいなサイン波が得られている

DDSの周波数の決まり方

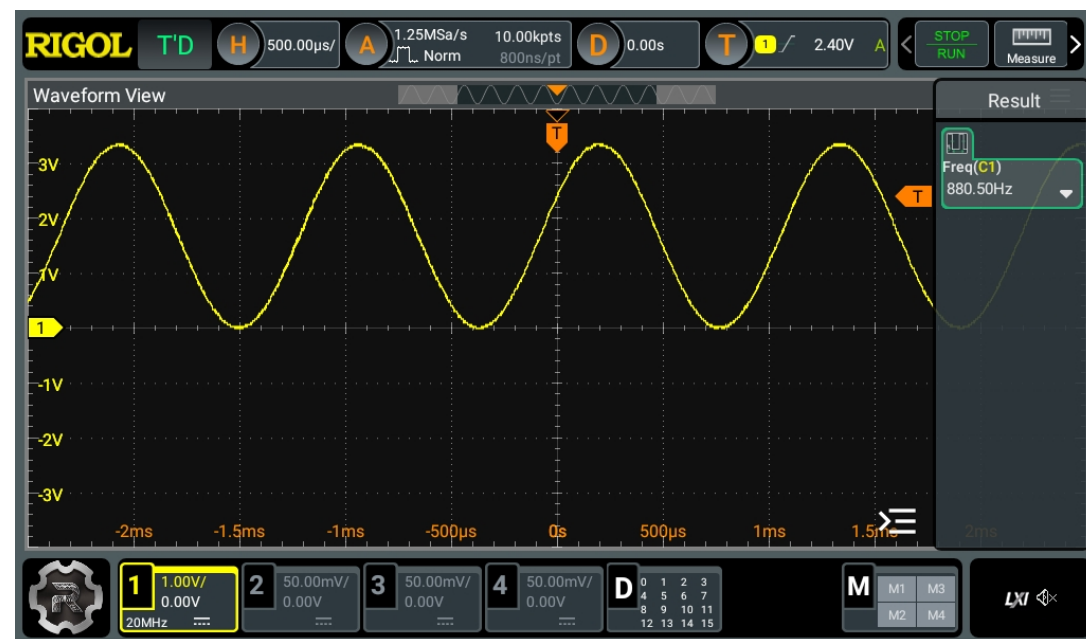
このDDSは1周期が24ビット



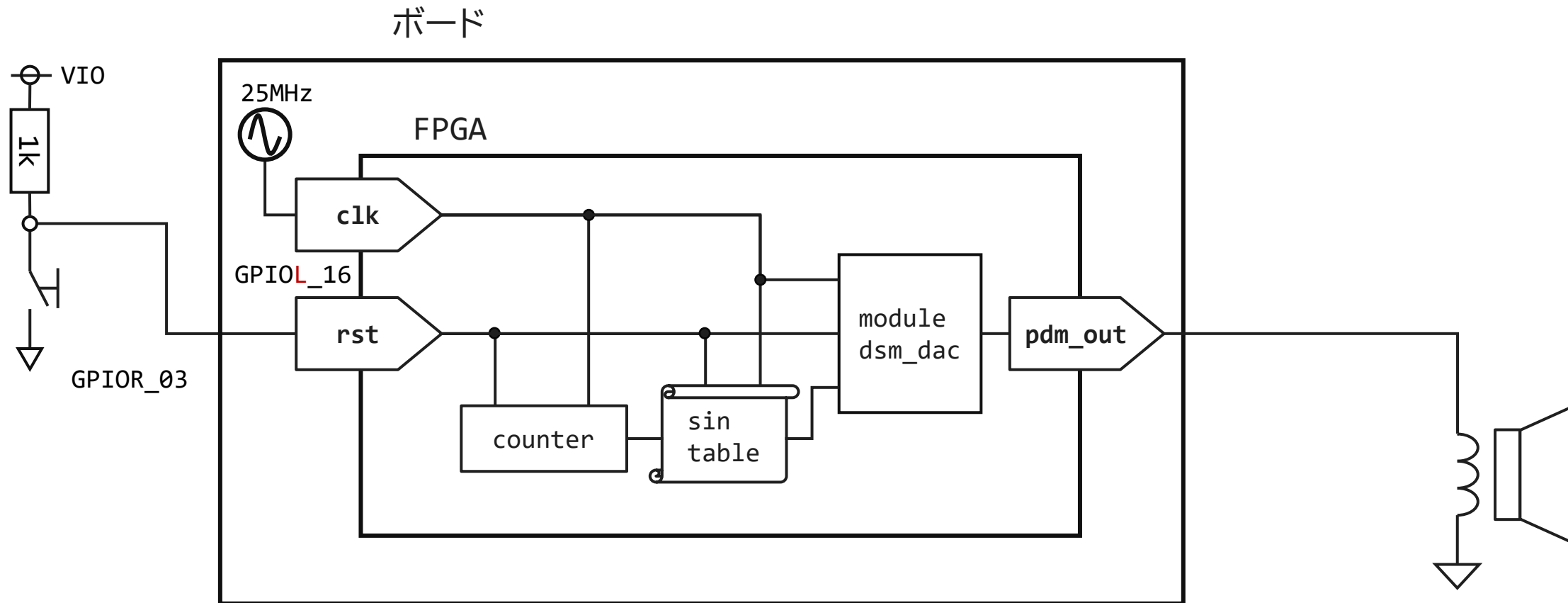
1周するには16,777,215カウント必要

25MHzで毎回1ステップ加算すると、
1秒間に25,000,000
→ $25,000,000 / 16,777,216 = 1.490 \text{ Hz/step}$
となる。

もし880Hz(ラの音)が欲しいときは...
 $880 / 1.490 = 591$ を指定する

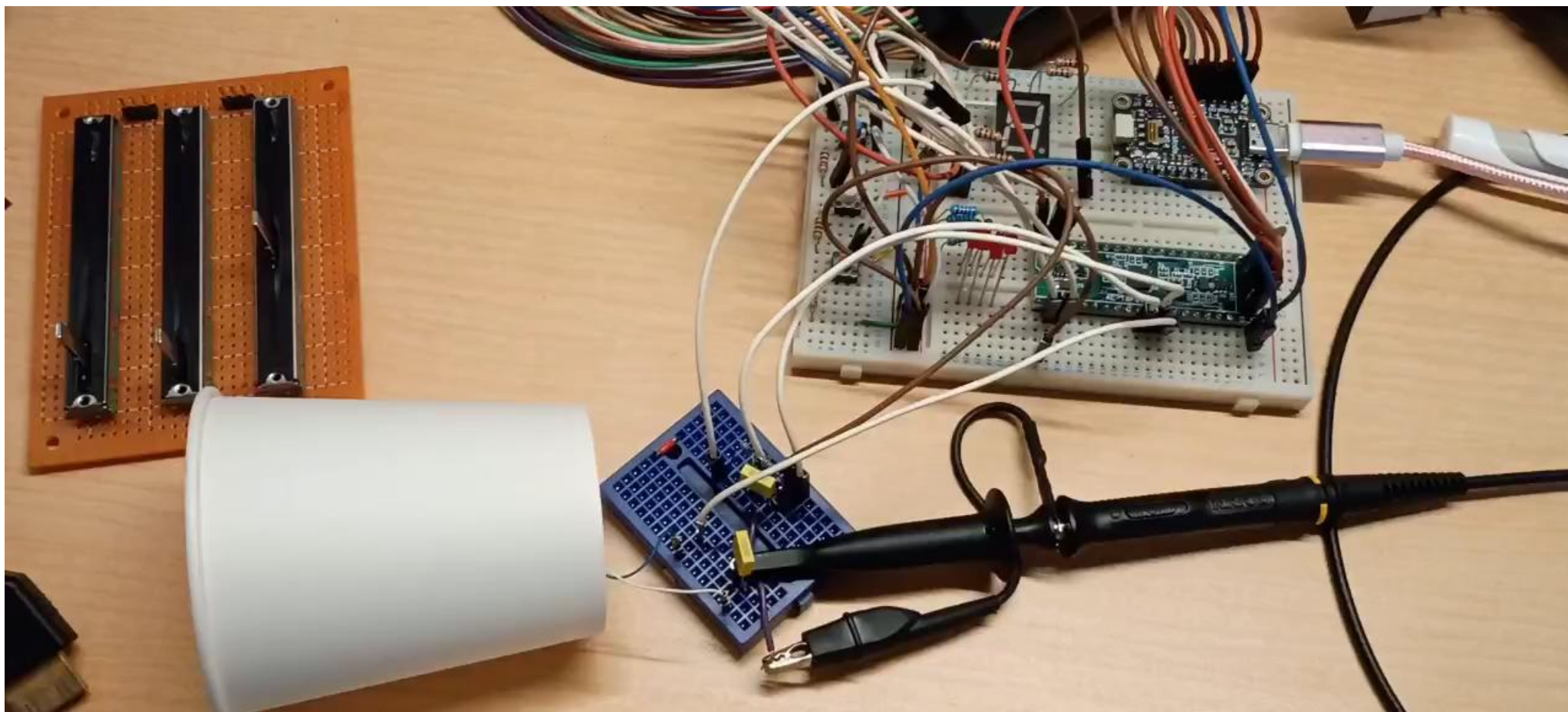


スピーカーで音を出してみる



スピーカは直接つなぐ。
(DCカットとアンプが欲しいが少しの間なら無くても大丈夫)

こんな音になる



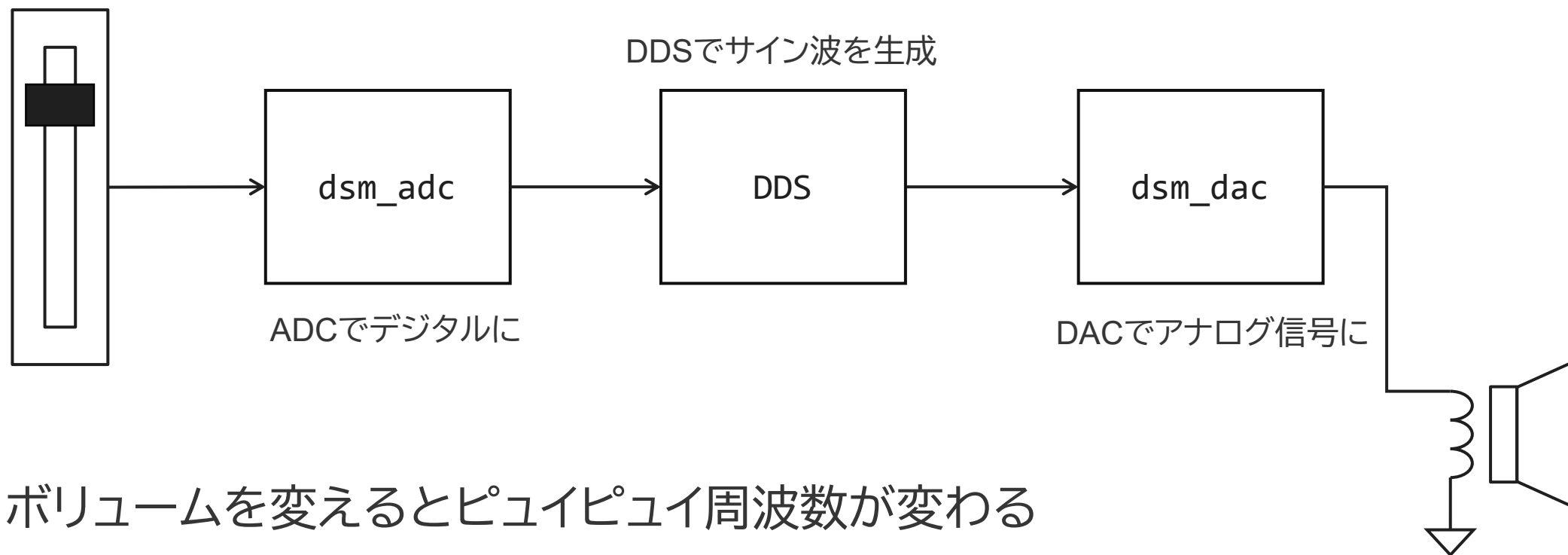
フィルタが無いので倍の1760Hzのほうがよく聞こえる

これまでの組み合わせ

$\Delta \Sigma$ -ADCと $\Delta \Sigma$ -DACを組み合わせてなにか作る

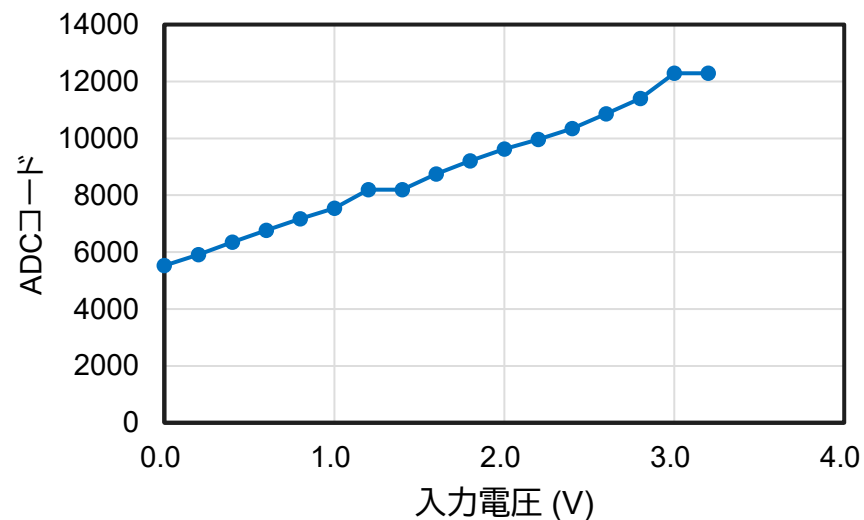
ボリュームを使ってシンセサイザを作る

ボリュームでアナログ信号を生成



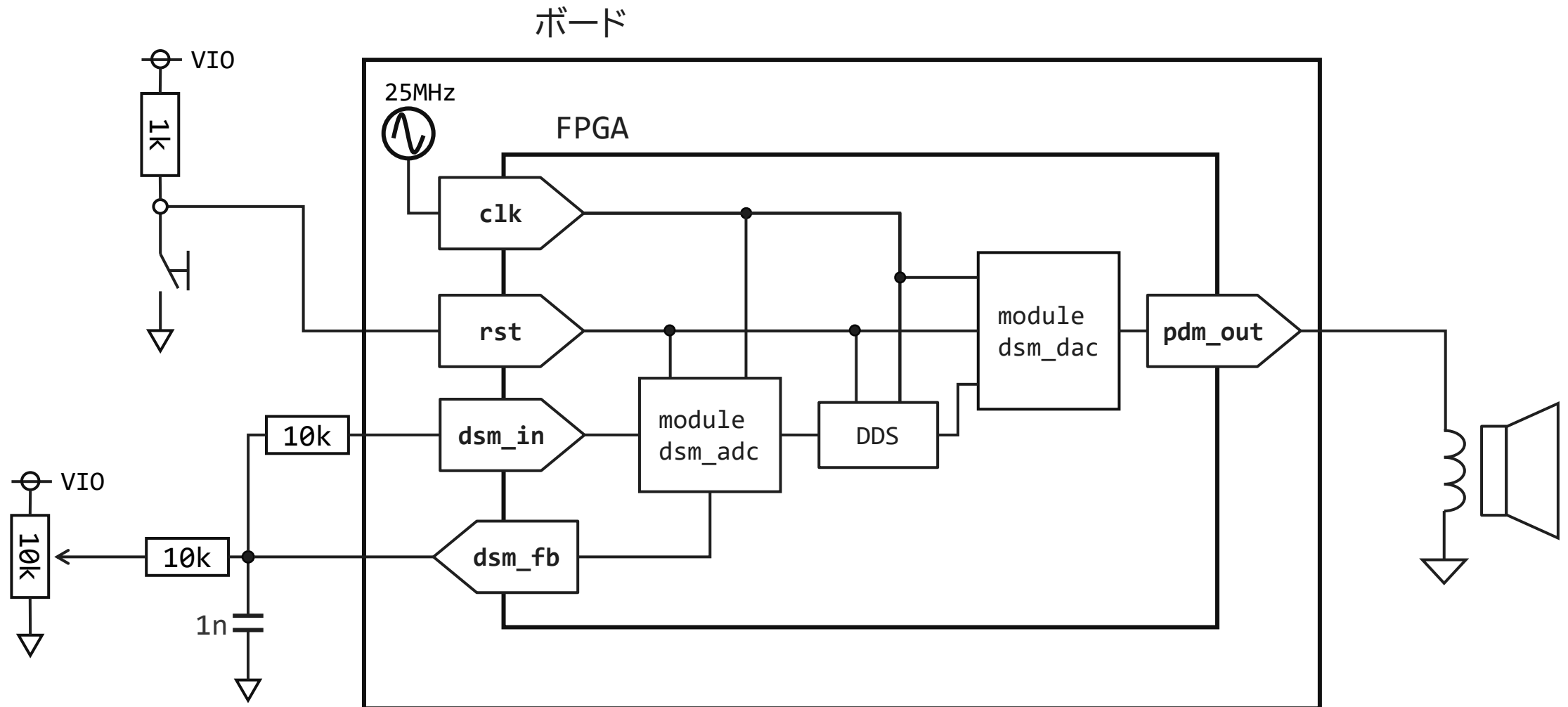
- スピーカーで鳴らす周波数範囲
 - 1600~3600Hz
- クロック周波数は5MHz
 - 25MHzのノイズが多いので下げる
- ボリュームの電圧可変範囲
 - 0~3.3V

→ クロック5MHzではDDSの値で5525~12288



ADC結果(上位13ビット)

上位13ビットを使えばそのまま入力できる



Top moduleの実装

```
1 module synth(  
2   input wire clk,  
3   input wire xrst,  
4   input wire dsm_adc_in,  
5   output wire dsm_adc_fb,  
6   output wire pdm_out);  
7
```

```
8   wire [15:0] omega;  
9
```

```
10  ⊖ dds_dac out_dac(  
11    .clk(clk),  
12    .xrst(xrst),  
13    .omega({11'd0, omega[15:3]}),  
14    .pdm_out(pdm_out));  
15
```

DDS(DAC入)のインスタンス

ADCの結果の上位13ビットを渡す

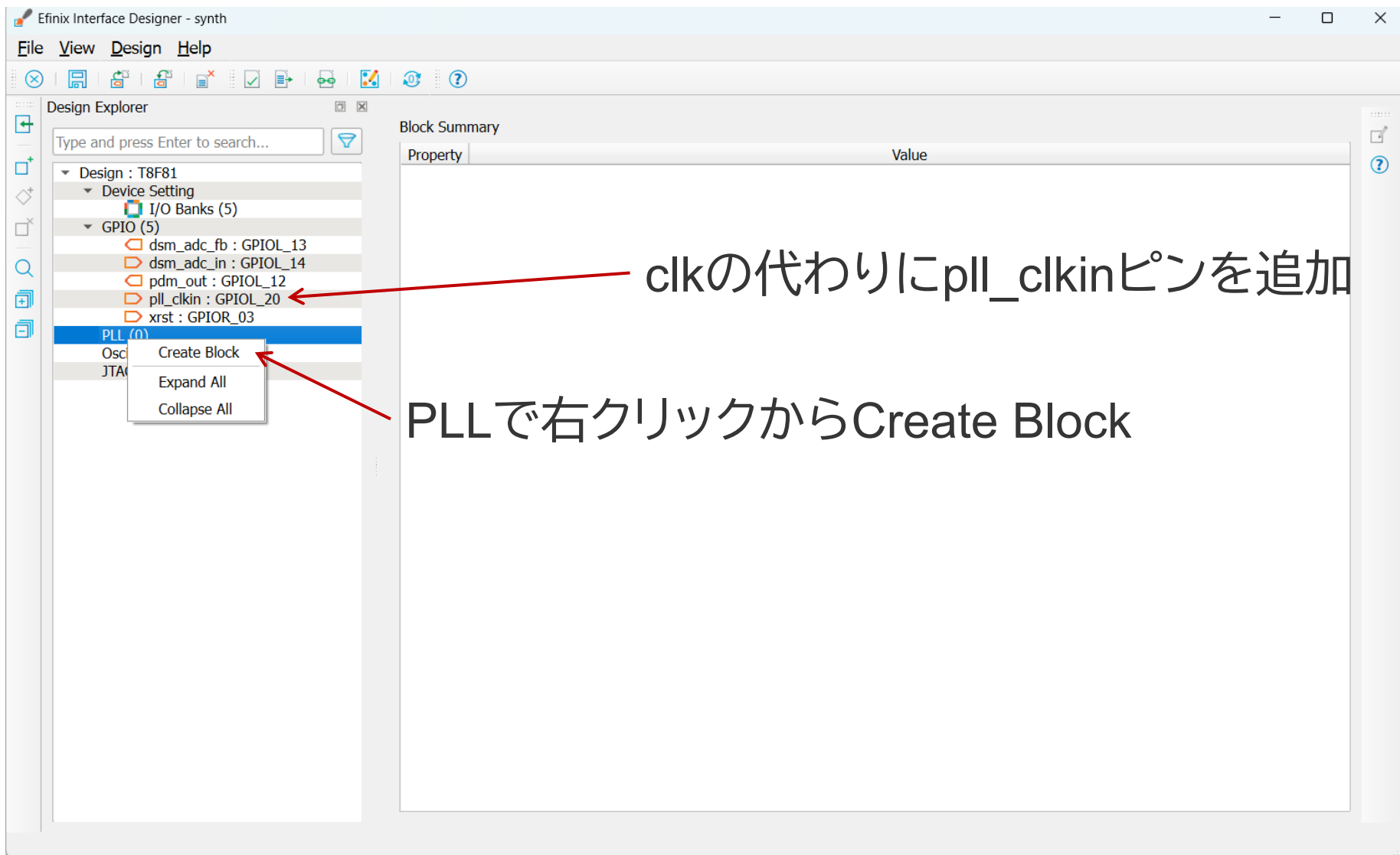
```
16  
17  ⊖ ctrl_adc in_adc(  
18    .clk(clk),  
19    .xrst(xrst),  
20    .dsm_in(dsm_adc_in),  
21    .dsm_fb(dsm_adc_fb),  
22    .code(omega));  
23
```

ADCのインスタンス

```
24  
25  endmodule  
26
```

PLLの設定方法1

clkを5MHzに設定する方法



PLLの設定方法2

The screenshot shows the Efinix Interface Designer - synth interface. The Design Explorer on the left lists the design components, with 'pll_inst1 : PLL_0' selected. The Block Summary table in the center lists the properties and values for the PLL instance. The Block Editor on the right shows the 'Properties' tab with the 'Instance Name' set to 'pll_inst1' and the 'PLL Resource' set to 'PLL_0'. A blue arrow points from the text '左クリック' (Left Click) to the 'Automated Clock Calculation' button at the bottom of the Block Editor.

Design Explorer

Type and press Enter to search...

- Design : T8F81
 - Device Setting
 - I/O Banks (5)
 - dsm_adc_fb : GPIOL_13
 - dsm_adc_in : GPIOL_14
 - pdm_out : GPIOL_12
 - pll_clkln : GPIOL_20
 - xrst : GPIOR_03
 - PLL (1)
 - pll_inst1 : PLL_0
 - Oscillator (0)
 - JTAG User Tap (0)

Block Summary

	Property	Value
1	Instance Name	pll_inst1
2	PLL Resource	PLL_0
3	Reference Clock Resource	GPIOL_20
4	Reference Clock Name	pll_clkln
5	Reset Pin Name	
6	Locked Pin Name	
7	Reference Clock Frequency (MHz)	10
8	Multiplier	80
9	Pre-Divider	1
10	VCO Frequency (MHz)	800.0000
11	Post-Divider	1
12	PLL Frequency (MHz)	800.0000
13	Output Clock 0	
14	Clock Pin Name	pll_inst1_CLKOUT0
15	Output Divider	8
16	Output Frequency (MHz)	100.0000

Block Editor

Properties Manual Configuration

Instance Name
pll_inst1

PLL Resource
✓ PLL_0

左クリック

Automated Clock Calculation

PLLの設定方法3

PLL Clock Calculator

Manual Mode: ☐ Feedback Mode: Internal ?

25を入力

Input Pin

Input Frequency
25.0000 MHz

Reset Pin

5を入力

Clock 0 Frequency
5.0000 MHz Actual: 4.9805 MHz

clk

clkを入力

Clock 1 Frequency
- MHz Actual: - MHz

Clock 2 Frequency
- MHz Actual: - MHz

最後にFinishを
左クリック

Cancel Finish

PLLの設定方法4

41

The screenshot shows the Efinix Interface Designer interface with the Resource Assigner, Design Explorer, Block Summary, and Block Editor panels.

Resource Assigner - GPIO : Instance View

Instance	Package Pin	Resource	I/O Bank	Alt Conn	Features	Clock Region	Pad
dsm_adc_fb	F1	GPIOL_13	1A	GCTRL	None	L0	GPIOL_13_CTRL1
dsm_adc_in	E2	GPIOL_14	1A	GCLK	None	L0	GPIOL_14_CLK0
pdm_out	G1	GPIOL_12	1A	GCTRL	None	L0	GPIOL_12_CTRL0
pll_clkln	C3	GPIOL_20	1B	PLL_CLKIN	None	L1	GPIOL_20_PLLIN
xrst	A6	GPIOR_03	2A	None	None	R1	GPIOR_03

Design Explorer

- Design : T8F81
 - Device Setting
 - I/O Banks (5)
 - GPIO (5)
 - dsm_adc_fb : GPIOL_13
 - dsm_adc_in : GPIOL_14
 - pdm_out : GPIOL_12
 - pll_clkln : GPIOL_20
 - xrst : GPIOR_03
 - PLL (1)
 - pll_inst1 : PLL_0
 - Oscillator (0)
 - JTAG User Tap (0)

Block Summary

Property	Value
1 Instance Name	pll_clkln
2 GPIO Resource	GPIOL_20
3 Mode	input
4 I/O Standard	3.3 V LVTTTL / LVCMOS
5 Unused State	NA
6 Alternate Connection	PLL_CLKIN
7 Features	None
8 Clock Region	L1

Block Editor

Input

I/O Standard: 3.3 V LVTTTL / LVCMOS

Pin Name: pll_clkln

Connection Type: pll_clkln

Register Option: none

Annotations:

- GPIO resource assignment table: GPIOL_20を指定 (Specify GPIOL_20)
- Design Explorer: pll_clklnを選択 (Select pll_clkln)
- Block Editor: Connection Typeを pll_clklnに変更 (Change Connection Type to pll_clkln)

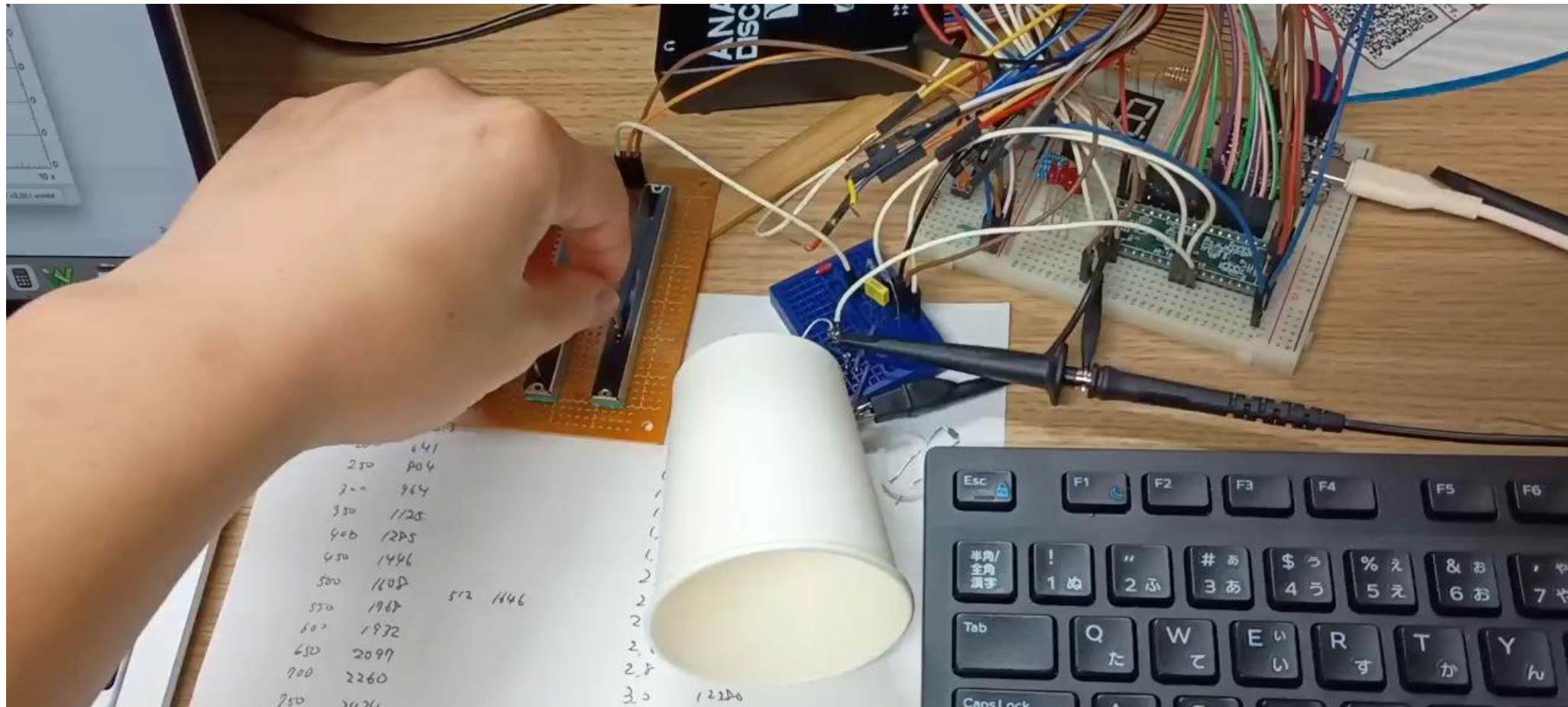
ピンアサイン

GPIO : Instance View

Instance	Package Pin	Resource	I/O Bank	Alt Conn	Features	Clock Region	Pad
 dsm_adc_fb	F1	GPIOL_13	1A	GCTRL	None	L0	GPIOL_13_CTRL1
 dsm_adc_in	E2	GPIOL_14	1A	GCLK	None	L0	GPIOL_14_CLK0
 pdm_out	G1	GPIOL_12	1A	GCTRL	None	L0	GPIOL_12_CTRL0
 pll_clkln	C3	GPIOL_20	1B	PLL_CLKIN	None	L1	GPIOL_20_PLLIN
 xrst	A6	GPIOR_03	2A	None	None	R1	GPIOR_03

結果

43



まとめ

- Verilogの文法を超特急でやって、なにかアプリケーションを実装するまでを体験してもらいました
- FPGAと少しのパッシブ部品でAD/DAを実現する $\Delta\Sigma$ 変調器を実装しました。
- DDSを実装して謎のシンセサイザを作成しました
- 外にダウンコンバータを付けられるとAMラジオも行ける(ような気がします)